

SAPIENZA
UNIVERSITÀ DI ROMA

Interactive Graphics

"Mission on Mars"

Giacomo Bernardini
2070434

2025/26

Contents

1 Introduction **2**
- 1.1 Overview . . . 2
- 1.2 Game Concept . . . 2
- 1.3 Gameplay Summary . . . 2

2 Development Environment and Libraries **3**
- 2.1 WebGL . . . 3
- 2.2 Three.js . . . 3
- 2.3 GLTFLoader . . . 3
- 2.4 External Assets . . . 3

3 Software Architecture **4**
- 3.1 Project Structure . . . 4
- 3.2 Application Flow . . . 5
- 3.3 Game State Management . . . 5
- 3.4 Asset Loading and Management . . . 5
- 3.5 Main Game Loop . . . 5

4 Scene Composition **5**
- 4.1 Scene Initialization and Rendering . . . 5
- 4.2 Dual Camera System . . . 5
- 4.3 Atmospheric Effects . . . 6
- 4.4 Terrain and Environment Layout . . . 6

5 Lighting and Rendering **7**
- 5.1 Dynamic Lighting System . . . 7
- 5.2 Materials and Localized Lights . . . 7

6 Hierarchical Models and Procedural Animations **7**
- 6.1 Complex Hierarchical Model: The Rover . . . 7
- 6.2 Forward Kinematics and Robotic Arm Control . . . 8
- 6.3 Alien Swarm Hierarchies and Flight Animation . . . 8
- 6.4 Environmental Animations and Battery Physics . . . 8

7 Gameplay Systems **9**
- 7.1 Objectives and Interaction . . . 9
- 7.2 Navigation and Radar System . . . 9
- 7.3 Combat, Hitboxes, and Shooting Mechanics . . . 9
- 7.4 Mathematical Collisions and Swarm AI . . . 10

8 User Interface **10**
- 8.1 Main Menu and State Overlays . . . 10
- 8.2 Heads-Up Display (HUD) . . . 10
- 8.3 Combat Damage, Victory, and Game Over Screens . . . 11

9 Conclusions **11**

1 Introduction

1.1 Overview

Mission on Mars is a browser-based 3D survival and exploration game developed using JavaScript and the Three.js graphics framework. The application places the player in control of a lone rover stranded on the desolate Martian surface, where survival depends on navigating hostile terrain, manipulating a robotic arm to gather energy cores, and transitioning into a combat phase to survive an alien swarm attack.

The project combines real-time rendering techniques, procedural animations, and interactive gameplay systems to create a fluid gaming experience executed entirely within a web browser.

1.2 Game Concept

The game is set in a desolate sci-fi scenario following a catastrophic malfunction of the Artificial Sun at a Martian extraction base. The player must guide a fully articulated rover through a continuous procedural sandstorm to locate scattered energy batteries. These cores are essential to reignite the Artificial Sun and restore both power and visibility to the environment.

The visual style focuses on a stylized, low-poly aesthetic enhanced by physically based rendering (PBR) materials. A dynamic lighting progression shifts the atmosphere from a dark, tension-filled dust storm illuminated only by the rover's headlights, to a bright, triumphant climax as the Artificial Sun is restored, setting both the narrative context and the visual identity of the experience.

1.3 Gameplay Summary

Mission on Mars blends vehicular exploration with fast-paced combat mechanics. Initially, the player operates in a third-person perspective, utilizing a custom driving model that reacts to the procedural terrain's elevation. The player must use a UI-controlled magnetic robotic arm to pick up energy batteries and deposit them into a central hopper, guided by a real-time radar system.

Once all batteries are successfully deposited, the game quickly transitions into a First-Person Shooter (FPS) perspective. A horde of hierarchical, voxel-styled alien mechas emerges from the sand, driven by a custom artificial intelligence algorithm. The player must defend the rover using a laser cannon equipped with a difficulty-scaled cooldown system and precise hitboxes.

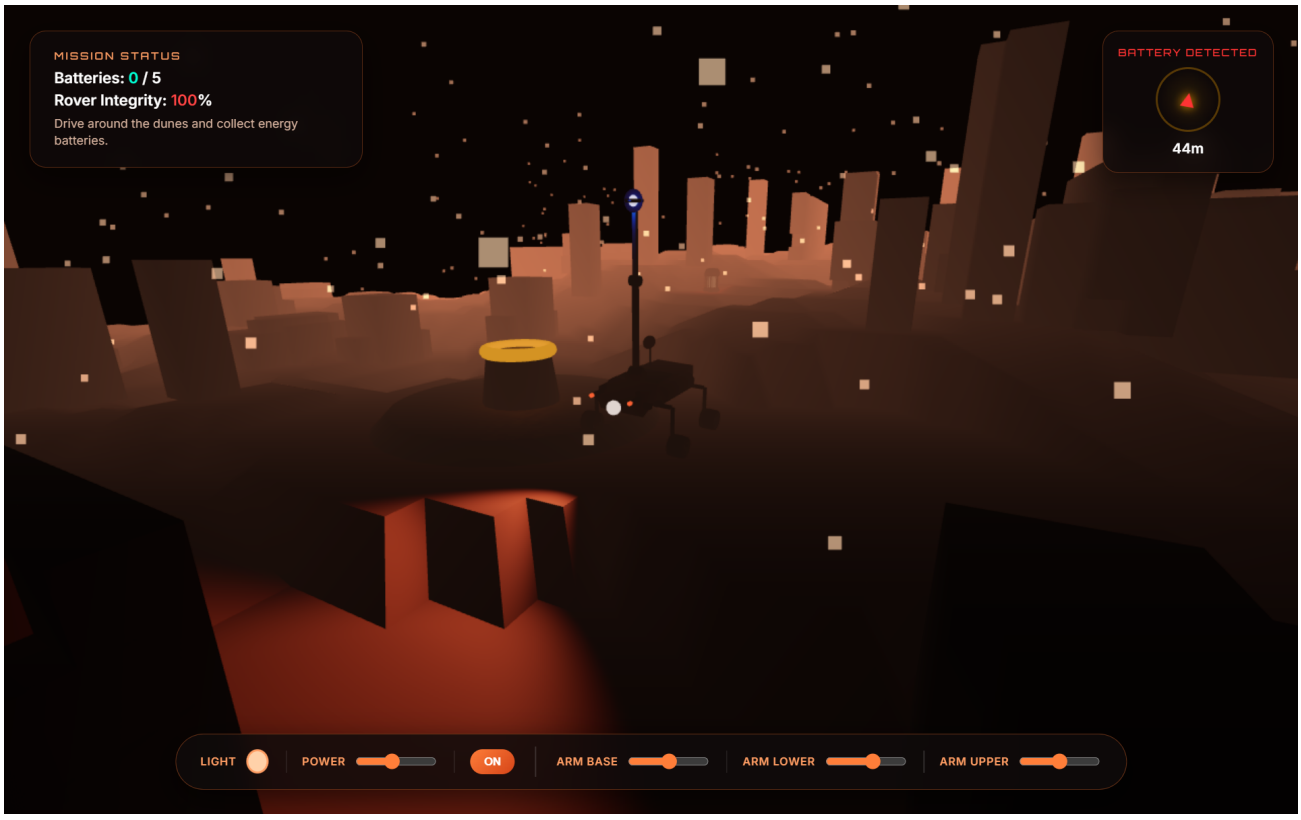


Figure 1: Third-person driving perspective showing the rover and the HUD.

2 Development Environment and Libraries

2.1 WebGL

Mission on Mars is developed as a browser-based application built upon WebGL. This standard graphics API allows for hardware-accelerated 3D rendering directly within modern web browsers. By utilizing WebGL, the application can render the 3D environment, process lighting, and handle the rendering pipeline in real-time without the need for external plugins or client-side installations.

2.2 Three.js

To manage the complexity of raw WebGL, the project utilizes Three.js, a lightweight and versatile 3D JavaScript library. Three.js acts as the core rendering engine of the application, providing an abstraction layer that simplifies the management of the scene graph, cameras, materials, and lighting systems.

In the context of this project, Three.js is fundamental for handling the transition of the environment (such as the lighting shift from the dark sandstorm to the reactivated Artificial Sun), managing the camera perspective switch between the third-person exploration phase and the first-person combat phase, and utilizing Raycasters specifically for vehicle suspension systems and precise hit-scan shooting mechanics.

2.3 GLTFLoader

To integrate external 3D models into the scene, the application relies on the GLTFLoader module provided by Three.js. This tool is designed to load `.gltf` and `.glb` files, which are the modern standard for 3D asset delivery on the web.

The `.glb` format was specifically chosen for its ability to efficiently pack geometry, node hierarchies, and material data into a single binary file. Through the GLTFLoader, some of the essential components of the game—such as the headlight, the wheels and the energy batteries—are asynchronously loaded during the initialization phase and instantiated into the main scene graph.

2.4 External Assets

While the core logic and environmental setup are developed programmatically, the visual and auditory fidelity of the main interactive entities is achieved using a selection of external resources. The following 3D models, textures, and audio files were explicitly integrated into the project:

- **Rover Wheels**: 3D model sourced from Sketchfab (`https://skfb.ly/6SHGI`).
- **Rover Headlights**: 3D model sourced from Sketchfab (`https://skfb.ly/ozuAp`).
- **Energy Battery**: 3D model sourced from Sketchfab (`https://skfb.ly/oWRzK`).
- **Rover Chassis Texture**: Corrugated iron PBR texture sourced from Poly Haven (`https://polyhaven.com/a/corrugated_iron_02`).
- **Base Structure Texture**: Rusty metal grid PBR texture sourced from Poly Haven (`https://polyhaven.com/a/rusty_metal_grid`).
- **Gameplay Music**: "Infinite Space" track sourced from Pixabay (`https://pixabay.com/it/music/elettronico-infinite-space-149523/`).
- **Menu/Game Over/Win Music**: "Dark Ambient" track sourced from Pixabay (`https://pixabay.com/it/music/ambiente-dark-ambient-126122/`).
- **Laser Sound**: Sound effect sourced from Pixabay (`https://pixabay.com/sound-effects/film-special-effects-laser-312360/`).
- **Alien Sound**: Sound effect sourced from Pixabay (`https://pixabay.com/it/sound-effects/film-ed-effetti-speciali-small-insect-monster-6-580961/`).
- **Alien Death Sound**: Sound effect sourced from Pixabay (`https://pixabay.com/it/sound-effects/film-ed-effetti-speciali-giant-insect-death-580992/`).

These visual resources are rendered using Physically Based Rendering (PBR) materials to ensure realistic light interaction within the Martian environment. Additionally, the integrated audio assets provide crucial auditory feedback for environmental atmosphere, combat actions, and game state transitions, anchoring the player's immersion during gameplay.

3 Software Architecture

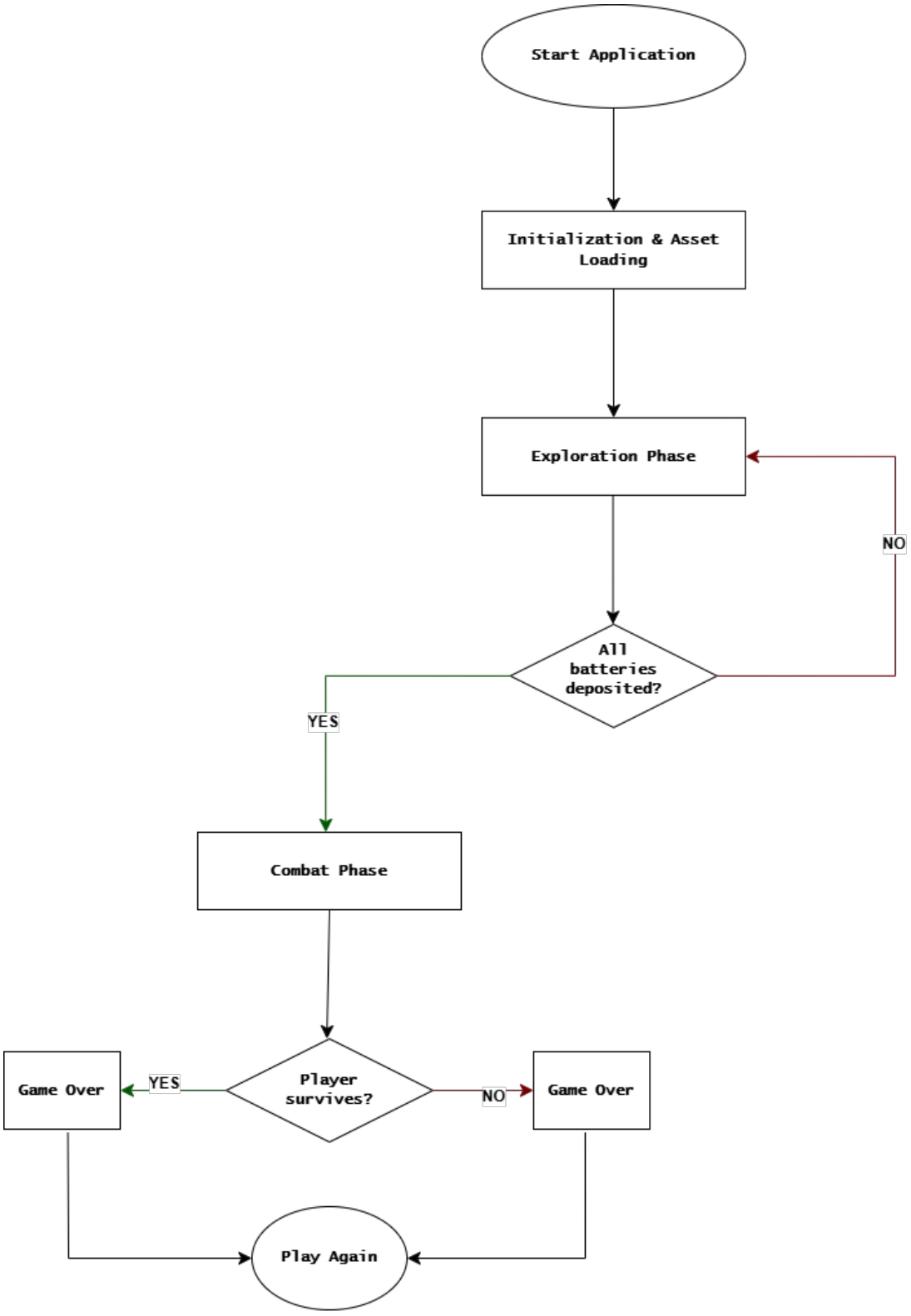


Figure 2: Overview of the application's modular structure and logic flow.

3.1 Project Structure

The application is organized using a modular JavaScript architecture. This approach separates the core rendering components from the game logic, user interface, and asset management. By dividing the project into distinct logical sections—such as scene initialization, input handling, physics updates, and entity management—the codebase remains maintainable, readable, and scalable, preventing logic overlap between the user interface and the 3D canvas.

3.2 Application Flow

The execution flow guides the player smoothly from the initial startup to active gameplay. When the application is launched, the fundamental Three.js components (WebGL renderer, scene, camera, and basic lighting) are initialized. An asset loading phase then fetches all required 3D models, textures, and audio files. Once loading is complete, the player enters the exploration phase. After successfully collecting and depositing the required energy cores, the application dynamically transitions into the combat phase. The flow concludes with either a victory screen (if the alien swarm is defeated) or a game over screen.

3.3 Game State Management

To avoid logic conflicts, the application uses a state-based approach managed through a centralized object containing boolean flags (e.g., `gameStarted`, `isGameOver`, `missionWon`) and condition trackers. The combination of these variables dynamically defines the primary gameplay phases: *Menu/Idle*, *Exploration*, *Combat*, and *Endgame (Victory/Game Over)*. The active phase dictates which functions are executed during the main loop. For example, rover driving mechanics and robotic arm controls are exclusively processed during the exploration phase, while alien pathfinding and shooting mechanics are only evaluated after all batteries are collected and the combat phase triggers.

3.4 Asset Loading and Management

To ensure a smooth gameplay experience without frame drops, all external resources (such as textures, audio files, the rover components, and the energy batteries) are managed through a centralized loading system. Using Three.js utilities such as GLTFLoader and TextureLoader, along with the native HTML5 Audio API, assets are imported asynchronously before the active gameplay begins. Once loaded, these base models and materials are stored in memory, allowing the application to quickly clone and instantiate them across the scene as needed.

3.5 Main Game Loop

The core of the application is driven by a real-time update loop implemented via the browser's `requestAnimationFrame` API. The `animate(time)` function evaluates the current game state to determine which logic blocks to execute. If the game is in an idle, Game Over, or Victory state, it strictly processes cinematic camera orbits or the extraction sequence. During active gameplay, it systematically executes physics updates, rover movement, robotic arm kinematics, battery magnet interactions, and alien AI pathfinding before instructing the renderer to draw the frame.

```
function animate(time) {
    requestAnimationFrame(animate);
    // 1. Cinematics and idle state execution
    if (!state.gameStarted || state.isGameOver || state.missionWon) {
        // Handle cinematic camera orbit or lift-off sequence
        // [...]
        renderer.render(scene, camera);
        return; // Halt further gameplay logic
    }
    // 2. Active Gameplay Logic
    // Process physics, input, collisions, and AI
    // [...]

    renderer.render(scene, camera);
}
```

Listing 1: Main animation loop demonstrating state-based logic execution.

4 Scene Composition

4.1 Scene Initialization and Rendering

The visual foundation of Mission on Mars is established by configuring a core Three.js `Scene`. To achieve a cinematic and realistic appearance, the `WebGLRenderer` is configured with antialiasing enabled to smooth geometric edges. Furthermore, physically accurate color representation is ensured by setting the output color space to sRGB and enabling ACES Filmic Tone Mapping. Shadow mapping is globally activated, allowing the dynamic light sources—such as the rover's headlights and the Artificial Sun—to cast realistic shadows from the environment and the alien entities.

4.2 Dual Camera System

A defining technical feature of the application is its dual camera system, which adapts to the current game state:

- **Exploration Phase (Third-Person):** A perspective camera is tethered to the rover. The camera calculates an offset vector relative to the rover's position and rotation, smoothly interpolating its movement to trail the vehicle across the uneven terrain.
- **Combat Phase (First-Person):** Upon depositing the final energy core, the camera transitions to a first-person perspective, simulating the viewpoint of a defense turret. The controls switch to pointer-lock mouse inputs, allowing precise aiming and shooting against the incoming alien swarm.

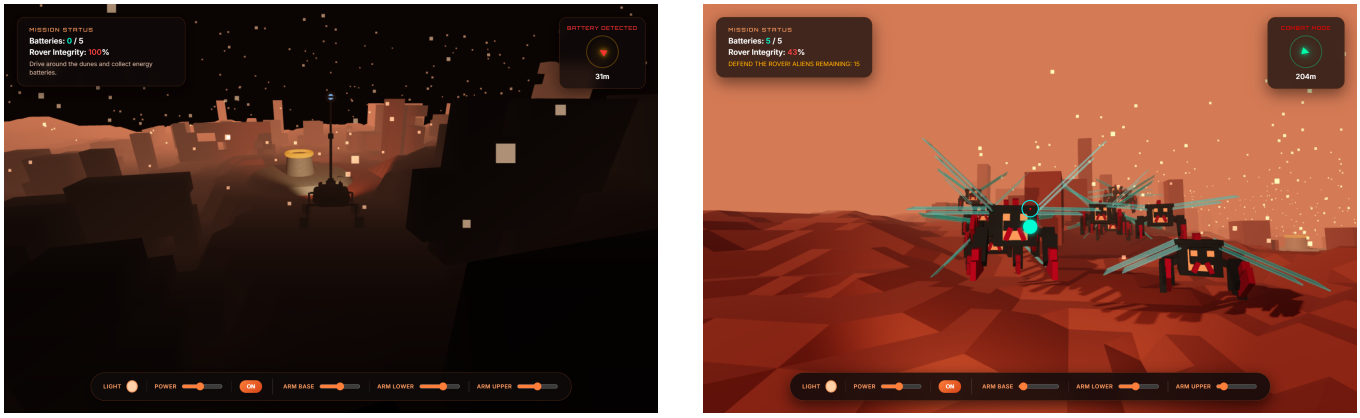


Figure 3: Left: Third-person exploration in the sandstorm. Right: First-person combat after the transition.

4.3 Atmospheric Effects

To immerse the player in the hostile Martian environment, the scene heavily relies on procedural atmospheric effects. A dense Three.js `FogExp2` (exponential fog) is applied to the scene, obscuring distant objects and naturally blending the terrain into the dark sky background. The sandstorm is simulated using a custom particle system created with `THREE.Points`, which dynamically updates thousands of dust particles around the player with minimal performance overhead. Once all the batteries have been identified, these atmospheric parameters are smoothly interpolated to reveal a crisp, well-lit environment.

4.4 Terrain and Environment Layout

The Martian surface serves as the foundation for both visual layout and physics calculations. Rather than importing a static terrain mesh, the environment is generated procedurally. A highly subdivided `PlaneGeometry` is created, and its individual vertex positions are mathematically displaced on the Y-axis. The elevation is calculated using a composite function of overlapping sine and cosine waves to create natural-looking hills and craters, while keeping specific objective areas flat.

```
export function getTerrainHeight(x, z) {
    // Composite waves for hills and craters
    let y = Math.sin(x * 0.015) * Math.cos(z * 0.015) * 12.0;
    y += Math.sin(x * 0.08) * Math.cos(z * 0.08) * 3.0;
    y += Math.sin(x * 0.25) * Math.cos(z * 0.25) * 1.0;

    // Apply a stepped/terraced look to the terrain
    return Math.floor(y / 0.8) * 0.8;
}

// Displace each vertex vertically based on the math function
const positionAttribute = floorGeo.attributes.position;
for (let i = 0; i < positionAttribute.count; i++) {
    const x = positionAttribute.getX(i);
    const z = positionAttribute.getZ(i);
    positionAttribute.setY(i, getTerrainHeight(x, z));
}
// Recalculate lighting normals after displacing vertices
floorGeo.computeVertexNormals();
```

Listing 2: Mathematical terrain generation modifying geometry vertices.

This procedural terrain is rendered with flat shading to achieve a stylized aesthetic. To populate this environment, the application procedurally generates over a hundred obstacle clusters—ranging from vertical pillars to archways and walls—using randomized geometric configurations that mathematically avoid spawning inside key objective areas.

Particle systems are utilized to further enrich the scene. The global sandstorm effect is implemented using a custom `Points` system (`THREE.BufferGeometry`). Thousands of dust particles are continually updated within the main

loop to simulate wind physics, shifting left and falling vertically to create an immersive, low-visibility environment. Additionally, a dynamic particle emitter generates interactive dust trails from the rover's rear wheels, with the emission rate scaled proportionally to the vehicle's current velocity.

5 Lighting and Rendering

5.1 Dynamic Lighting System

The illumination of the Martian environment relies on a dynamic lighting system designed to enhance both atmosphere and user interaction. During the exploration phase, global illumination is heavily suppressed to simulate a dense sandstorm. The primary light sources are the rover's headlights, implemented using Three.js `SpotLight` objects attached directly to the vehicle's hierarchical chassis node. The player interacts with the scene by steering the rover, sweeping these headlights across the environment to cast real-time shadows and reveal hidden obstacles.

The most significant lighting event is tied to gameplay progression. Instead of an instantaneous switch, the application mathematically scales the ambient and directional lights based on the ratio of collected batteries. Furthermore, it interpolates the fog and background colors from a dark red to a brighter hue, completely transforming the visual environment in real-time as the Artificial Sun reactivates.

```
// Update lights based on collection progress
const progress = state.collectedBatteries / state.totalBatteries;
ambientLight.intensity = 0.05 + (maxAmbientIntensity * progress);
sunLight.intensity = maxSunIntensity * progress;

// Lerp fog and sky color from dark red to brighter
const r = THREE.MathUtils.lerp(0x0a, 0xd4, progress);
const g = THREE.MathUtils.lerp(0x04, 0x7a, progress);
const blue = THREE.MathUtils.lerp(0x02, 0x55, progress);

scene.background.setHex((Math.floor(r) << 16) | (Math.floor(g) << 8) | Math.floor(blue));
scene.fog.color.setHex((Math.floor(r) << 16) | (Math.floor(g) << 8) | Math.floor(blue));
```

Listing 3: Dynamic scaling of lights and fog interpolation based on gameplay progress.

5.2 Materials and Localized Lights

To achieve high visual fidelity, the project utilizes Physically Based Rendering (PBR) through the `MeshStandardMaterial`. The application strictly implements multiple types of texture maps to define surface properties. Specifically, the imported materials for the rover and the base structures utilize:

- **Color (Diffuse) Maps:** To define the base pigmentation and rusted metal look.
- **Normal Maps:** To simulate complex geometric details, such as corrugated iron grooves, without increasing polygon count.
- **Metalness/Roughness Maps:** To dictate how dynamic lights reflect off metallic chassis and dusty surfaces.

To complement the global lighting, the game heavily relies on localized `PointLight` objects and emissive textures to guide the player. Interactive elements—such as the scattered energy batteries, the robotic electromagnet, and the alien cores—are illuminated using these point lights and materials with high `emissiveIntensity`. This local illumination interacts naturally with the PBR textures, physically anchoring the entities to the procedural terrain while primary light sources handle the shadow casting.

6 Hierarchical Models and Procedural Animations

A core technical constraint of this project was the exclusive use of programmatic animations. No pre-baked or imported keyframe animations were used. All movements, mechanical joints, and entity behaviors were implemented entirely via code in JavaScript using Three.js mathematical utilities, deeply exploiting the hierarchical structure of the models.

6.1 Complex Hierarchical Model: The Rover

The rover is a complex hierarchical model organized programmatically. To simulate a realistic suspension system, downward-facing `Raycaster` objects continuously read the terrain's elevation geometry for each wheel. The telescopic forks adaptively scale, and the wheels adjust their vertical position using linear interpolation (`lerp`) based on the ground intersection points.

Additionally, the rover's movement speed is algorithmically adjusted to react to the terrain slope. By reading the chassis' X-axis rotation, the engine applies a "slope resistance" factor: velocity decreases significantly when climbing steep dunes and accelerates when navigating downhill.

```
// Cast a ray downwards from an anchor above each wheel
rayOrigin.copy(anchor);
rayOrigin.y += 15;
raycaster.set(rayOrigin, new THREE.Vector3(0, -1, 0));

const intersects = raycaster.intersectObjects([floor, depositStationGroup, extractionPointGroup],
    true);

if (intersects.length > 0) {
    const groundY = intersects[0].point.y;
    // [...]
    const targetWheelY = targetWorldPos.y;

    // Soft movement of the wheel
    wheel.position.y = THREE.MathUtils.lerp(wheel.position.y, targetWheelY, 0.2);

    // Fork dynamics: compress or extend the suspension visually
    const forkTopY = -0.4;
    const forkBottomY = wheel.position.y;
    const distance = Math.abs(forkTopY - forkBottomY);
    fork.scale.y = THREE.MathUtils.lerp(fork.scale.y, Math.max(0.1, distance), 0.3);
    fork.position.y = (forkTopY + forkBottomY) / 2;
}
```

Listing 4: Algorithmic suspension system utilizing Raycasters and linear interpolation.

6.2 Forward Kinematics and Robotic Arm Control

Attached to the rover's chassis is a fully articulated robotic arm, constructed as a multi-node hierarchical chain (`armBase` → `lowerPivot` → `upperPivot` → `magnetGroup`). The movement relies on forward kinematics, where the rotation of a parent node mathematically cascades down to all child nodes, preserving their local relative positions. The player manipulates these joints in real-time via the UI sliders.

6.3 Alien Swarm Hierarchies and Flight Animation

The voxel-styled alien mechas are constructed strictly as hierarchical models (`THREE.Group`) with interconnected nodes for the thorax, head, wings, arms, and legs. During the combat phase, their movement is a complex procedural flight animation driven by time elapsed. By applying oscillating trigonometric functions (`Math.sin` and `Math.cos`) combined with the entity's spatial coordinates, the alien's main body hovers, its glowing core pulsates and rotates, and its wings perform a continuous flapping cycle entirely computed by the CPU.

```
// Hovering animation for the main body
alien.body.position.y = 2.5 + Math.sin(time * 0.003 + alien.mesh.position.x) * 0.6;

// Rotation and pulsation of the internal core
if (alien.core) {
    alien.core.rotation.y = time * 0.002;
    alien.core.rotation.x = time * 0.001;
    alien.core.scale.setScalar(1.0 + Math.sin(time * 0.008) * 0.1);
}
//[...]
// Procedural wing flapping based on time and position offset
const flapAngle = Math.sin(time * 0.015 + alien.mesh.position.x) * 0.8;
const pitchAngle = Math.cos(time * 0.015 + alien.mesh.position.x) * 0.2;
if (alien.leftWing) alien.leftWing.rotation.set(pitchAngle, 0, flapAngle);
if (alien.rightWing) alien.rightWing.rotation.set(pitchAngle, 0, -flapAngle);
```

Listing 5: Procedural flight and wing flapping animations using trigonometric functions.

6.4 Environmental Animations and Battery Physics

Secondary physics-based animations are applied to environmental elements. The energy batteries are subjected to a custom physics simulation: by calculating the horizontal distance to the extraction hopper and reading the terrain height (`getTerrainHeight`), dropped batteries realistically fall via gravity until they hit the algorithmic ground. If dropped on the rim of the hopper, a push force is calculated using directional vectors to simulate a bouncy physical collision.

7 Gameplay Systems

7.1 Objectives and Interaction

The primary objective during the exploration phase is to collect scattered energy batteries. The total number of required batteries, along with their maximum procedural spawn radius from the central base, proportionally scales based on the player's selected difficulty level.

Interaction relies on vehicle positioning and robotic arm manipulation. The end-effector (magnet) continually calculates its spatial distance to the nearest battery. Once the distance threshold is met and the magnet is active, the battery's position smoothly interpolates (`lerp`) towards the magnet. Releasing the battery over the central hopper triggers a specific animation sequence that deposits the core, updates the game state, and triggers the global lighting transition.

7.2 Navigation and Radar System

To assist the player in the dense sandstorm, a real-time radar UI provides intuitive directional and distance feedback. The logic mathematically projects the 3D coordinates of the target (either a battery, the central base, or the final extraction pad) relative to the rover and the camera's current azimuthal angle. By utilizing the `Math.atan2` function, the system calculates the exact rotation required for the 2D HTML compass arrow to point towards the objective.

```
// Calculate distance and angle to the target
if (targetPos) {
    let dx = targetPos.x - roverGroup.position.x;
    let dz = targetPos.z - roverGroup.position.z;
    dom.radarDistance.textContent = Math.floor(Math.sqrt(dx*dx + dz*dz)) + "m";
    let targetAngle = Math.atan2(dx, dz);
    let camAngle = controls.getAzimuthalAngle();
    let rotation = (camAngle - targetAngle) * (180 / Math.PI) + 180;
    dom.radarArrow.style.transform = `translate(-50%, -50%) rotate(${rotation}deg)`;
}
```

Listing 6: Radar trigonometric logic to calculate UI arrow rotation and distance.

7.3 Combat, Hitboxes, and Shooting Mechanics

Upon triggering the combat phase, gameplay shifts smoothly to a First-Person Shooter perspective, utilizing the browser's Pointer Lock API for precise mouse-driven aiming. The shooting mechanics rely on a `Raycaster` originating from the camera's center to evaluate intersections against the aliens' hierarchical child meshes.

To introduce tactical depth, the system utilizes Three.js `userData` to create locational hitboxes: a direct hit to the designated head mesh neutralizes the target, while striking armor (such as wings, the body, or arms) solely provides visual UI feedback without destroying the entity.

Conversely, the alien swarm deals damage through a spatial proximity threshold. If an enemy breaches the rover's defensive perimeter, it continuously drains the vehicle's integrity score, triggering a Game Over state if reduced to zero.

Finally, a successful headshot removes the defeated entity from the active AI array and transfers it to a dedicated animation queue. The main loop then mathematically computes a gravity-driven backward fall. Once the entity intersects the procedural terrain elevation, it despawns and triggers a voxel-style particle burst, with outward trajectories and fading opacities calculated strictly frame-by-frame.

```
const alienMeshes = aliens.map(a => a.mesh);
    const intersects = shootRaycaster.intersectObjects(alienMeshes, true);

    if (intersects.length > 0) {
        endPos.copy(intersects[0].point); // stop the ray
        const hitMesh = intersects[0].object;
        let alienIdx = aliens.findIndex(a => {
            let p = hitMesh;
            while(p) { if(p === a.mesh) return true; p = p.parent; } return false;
        });

        if (alienIdx > -1) {
            // Check if the specific mesh we hit is the HEAD
            // Kill the aliens
            if (hitMesh.userData.isHead) {
                // remove from AI array
                const deadAlienData = aliens.splice(alienIdx, 1)[0];

                const dSound = deathSound.cloneNode();
                dSound.volume = 0.8;
                dSound.play().catch(e => console.log("error death audio:", e));
```

```
                // dying animation array
                dyingAliens.push({
                    mesh: deadAlienData.mesh,
                    fallSpeed: 0,
                    targetRotX: deadAlienData.mesh.rotation.x - (Math.PI / 2)
                });

                dom.missionStatus.innerHTML = `<span style="color:#ffaa00">DEFEND THE ROVER! ALIENS
                    REMAINING: ${aliens.length}</span>`;

                if (aliens.length === 0) {
                    dom.missionStatus.innerHTML = `<span style="color:#00ffd5">AREA CLEARED! PROCEED
                         TO EXTRACTION PAD.</span>`;
                }
            } else {
                // Armor hit (Body, Wings, Arms)
                dom.radarLabel.textContent = "ARMOR HIT! AIM FOR THE RED HEAD!";
                // [...]
            }
        }
    }
```

Listing 7: Raycaster shooting logic evaluating specific mesh hitboxes.

7.4 Mathematical Collisions and Swarm AI

Rather than relying on expensive physics engines, physical interactions are strictly managed through mathematical radial collision detection. By calculating the Euclidean distance (`Math.sqrt(dx*dx + dz*dz)`) between entity coordinates and bounding radii, the system efficiently handles terrain boundaries, battery drops, and interactions without complex mesh colliders.

Building upon this mathematical foundation, the alien entities are governed by a custom artificial intelligence loop featuring three core behaviors:

- **Obstacle Avoidance:** The AI continuously calculates its distance to the nearest procedural obstacle. If a collision is imminent, it computes the tangent vector (`tanX, tanZ`) to smoothly slide around the bounding radius rather than getting stuck.
- **Separation:** Entities continuously check spatial distances against their peers, applying a repelling force if they get too close (`odist < 3.5`) to prevent overlapping models.
- **Flanking Maneuvers:** Based on their array index, aliens apply alternating lateral forces to encircle the rover, creating a frenetic and highly threatening combat scenario.
- **Distance Control (Braking):** To prevent overlapping the player's vehicle, the AI calculates the dot product of its movement vector when within striking distance (`dist < 4.0`), smoothly decelerating to hold a threatening perimeter.

Finally, the combat difficulty dynamically scales based on the player's initial menu selection. Higher difficulty levels mathematically increase both the total spawn count of the swarm and the base movement speed of the AI, while simultaneously increasing the laser cannon's cooldown time, challenging the player's reaction times until the victory condition is met.

8 User Interface

8.1 Main Menu and State Overlays

The application features a comprehensive HTML and CSS-based user interface layered directly over the Three.js WebGL canvas. The experience begins with a Main Menu that establishes the narrative context of the mission and allows the player to initialize the game. By decoupling the 2D interface from the 3D rendering engine, the UI elements remain crisp, accessible, and responsive across different screen sizes, independent of the WebGL rendering resolution.

8.2 Heads-Up Display (HUD)

During active gameplay, a dynamic Heads-Up Display provides the player with critical real-time information, automatically adapting to the current game phase:

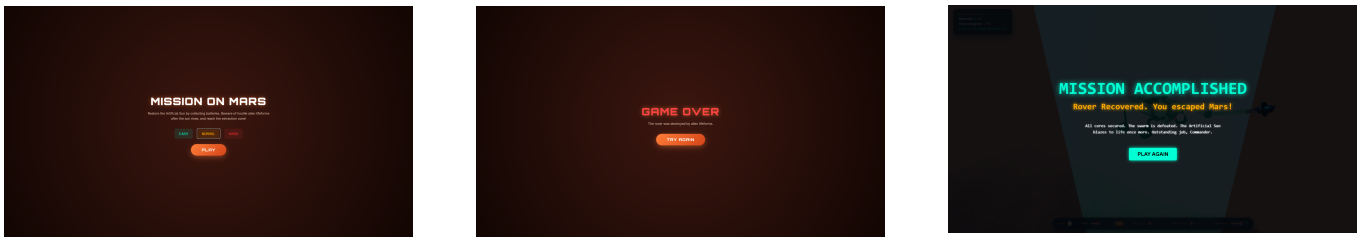
- **Exploration HUD:** Displays a progress counter for the collected energy batteries and the interactive radar compass that guides the player through the sandstorm. It also provides visual overlays for the rover's speed and the robotic arm's status.

- **Interactive Controls:** Beyond passive information, the HUD integrates interactive HTML sliders and toggles. These DOM elements directly manipulate Three.js properties in real-time, allowing the player to manually control the robotic arm's kinematic joints, toggle the rover's headlights, and manually configure the light's color and intensity.
- **Combat HUD:** Upon transitioning to the First-Person perspective, the UI clears unnecessary exploration elements and renders a central crosshair for precise aiming. It features a cooldown indicator for the laser cannon and a real-time health counter tracking the rover's structural integrity, providing immediate feedback on weapon readiness and survival status.

8.3 Combat Damage, Victory, and Game Over Screens

The application includes dedicated endgame interfaces that halt the gameplay loop upon reaching a terminal state. If the alien swarm drains the integrity of the rover and the value drops to zero, a Game Over screen appears. Conversely, if the player successfully eliminates the swarm and reaches the extraction pad, the audio track immediately changes and a cinematic sequence is triggered: the rover automatically aligns itself in the center of the platform, an extraction beam fades in from the sky, and the vehicle slowly ascends into it while spinning. Following this sequence, the Victory screen is displayed.

Both Victory and Game Over screens offer a clean restart functionality that safely reloads the browser context, initializing a fresh session for the player.



Main Menu Game Over Victory

Figure 4: User interface overlays across different game states.

9 Conclusions

The implementation of *Mission on Mars* highlights how complex 3D interactions can be managed natively in a browser without relying on heavy physics engines. By designing custom mathematical solutions for procedural terrain generation, raycast-based vehicle suspension, and radial collision detection, the project achieved a lightweight and highly responsive environment.

The structural shift from a third-person exploration phase to a first-person combat scenario proved to be an effective way to implement and test different control schemes, UI overlays, and camera perspectives within the same continuous application.

In conclusion, the project met the required technical constraints by prioritizing algorithmic logic and programmatic animations over pre-baked assets. This approach ensured a well-optimized, self-contained WebGL experience that successfully balances visual fidelity with real-time performance.